Software Requirements Specification

for

SDNScope

Version 1.1 approved

Prepared by
Sudhanva Y K (Developer)
Suhana A Ghosh (Developer)

Computer Networks Project (cn_project)

`https://github.com/sudhanvayk/cn_project`

November 2026

Document structure based on the IEEE 830 template by Karl E. Wiegers.
Permission is granted to use, modify, and distribute this template.

Table of Contents

1 Introduction **4**
1.1 Purpose 4
1.2 Document Conventions 4
1.3 Intended Audience and Reading Suggestions 4
1.4 Product Scope 4
1.5 References 5
2 Overall Description **5**
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Classes and Characteristics 6
2.4 Operating Environment 6
2.5 Design and Implementation Constraints 6
2.6 User Documentation 7
2.7 Assumptions and Dependencies 7
3 External Interface Requirements **7**
3.1 User Interfaces 7
3.2 Hardware Interfaces 8
3.3 Software Interfaces 8
3.4 Communications Interfaces 8
4 System Features **8**
4.1 Active Data-Plane Probing and Telemetry 9
4.2 Topology, Path, and Flow Discovery 9
4.3 Correlation, Health Classification, and Localization 10
4.4 Backup Path Selection and Rerouting 10
4.5 Reporting 11
5 Other Nonfunctional Requirements **11**
5.1 Performance Requirements 11
5.2 Safety Requirements 11
5.3 Security Requirements 11
5.4 Software Quality Attributes 12
5.5 Business Rules 12
6 Other Requirements **12**
Appendix A: Glossary **12**
Appendix B: Analysis Models **13**
Appendix C: To Be Determined List **13**
Appendix D: Requirement Traceability **13**

Revision History

Name	Date	Reason for Changes	Version
Sudhanva Y K	2026-10-19	First draft of the SDNScope requirements.	0.9
Sudhanva Y K	2026-10-25	Added live Mininet verification and reroute details.	1.0
Sudhanva Y K	2026-11-05	Final review: corrected the diamond topology, updated the supported Python range and documentation list, and added the qdisc-based localization approach.	1.1

1. Introduction

1.1 Purpose

This document describes what SDNScope is expected to do and how it should behave. It covers the complete tool: the UDP probe client and server, the controller topology collector, the OpenFlow flow-table inspector, the correlation engine, the reroute helper, and the report generator.

The document is a requirements document, not a design document. It says what the system must do and what limits it must respect. The internal design is described separately in `docs/DESIGN.md`.

1.2 Document Conventions

The document follows the IEEE 830 structure. A sentence that contains the word “shall” is a requirement. Requirements are labelled `REQ-1`, `REQ-2`, and so on so that they can be traced to tests.

Names of files, commands, and JSON fields are written in a monospace font, for example `correlate.py` or `correlation_report.json`. Percentages and times are written exactly as they appear in the captured results.

1.3 Intended Audience and Reading Suggestions

The document is written for three groups of readers:

- **Evaluators and instructors** can read Section 1.4 and Section 2 first. These sections explain the problem, the scope, and the overall behaviour without requiring knowledge of the source code.
- **Developers** should read Section 3 and Section 4. These sections describe the interfaces and the required behaviour of every functional area.
- **Testers** should read Section 4, Section 5, and the separate test plan (`software_test_plan_for_sdnscope.tex`). The test plan maps these requirements to concrete test cases and evidence.

1.4 Product Scope

A normal `ping` or `traceroute` tells you whether packets reached the other side, but it cannot see the OpenFlow rules that decide the path. In an SDN, the controller knows the topology, the flow tables, and the port counters, but it does not measure the actual quality of the path that the application is using. Troubleshooting therefore needs both views at the same time.

SDNScope combines the two views. It sends timestamped UDP probes between hosts, collects topology and flow information from the Ryu controller, and compares the two data sets. From that comparison it can say whether a path is healthy, degraded, or failed, which link is the most likely cause, and which backup path can be used. When a failure is confirmed, the reroute helper can install backup OpenFlow rules.

The tool is intended for a lab or teaching environment, mainly the three-switch diamond topology built with Mininet. It is not a production monitoring platform. It does not replace the controller, and it does not try to handle every possible network design. Its value is that every conclusion is backed by a measured number or a captured controller response, and every run is saved as evidence.

1.5 References

1. Project repository: `https://github.com/sudhanvayk/cn_project`. This contains the source code, the JSON schemas, and the captured test evidence.
2. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
3. OpenFlow Switch Specification, version 1.3, Open Networking Foundation.
4. Ryu SDN Framework documentation, including the `ofctl_rest` and topology REST APIs.
5. Mininet documentation, including `TCLink`, `OVSSwitch`, and the Python topology API.
6. Project documents: `README.md`, `SETUP.md`, `docs/DESIGN.md`, `docs/schemas.md`, `docs/EVIDENCE.md`, `TEST_CASES.md`, and `SUBMISSION_CHECKLIST.md`.

2. Overall Description

2.1 Product Perspective

SDNScope is a command-line toolkit that runs next to a small OpenFlow network. The network used for development and testing is a diamond built in Mininet:

```
      h1 (10.0.0.1)                 h2 (10.0.0.2)
           |                             |
          s1 (dpid 1) ---------------- s2 (dpid 2)
           \                           /
            \                         /
             \                       /
              +------ s3 (dpid 3) --+

Primary path:      h1 -> s1 -> s2 -> h2          (direct s1--s2 link)
Backup path:       h1 -> s1 -> s3 -> s2 -> h2    (via s3, one hop longer)
```

The primary route uses the direct `s1--s2` link. The backup route goes through `s3` and adds one hop. The switches are managed by Ryu over OpenFlow 1.3, and the hosts send UDP probes to each other.

For work that does not need root access, the repository also includes a localhost mode. In this mode `mock_controller.py` returns controller-like JSON, and the probe server and client run on the loopback interface. This is the mode used by the automated tests and by `make demo`.

2.2 Product Functions

The tool performs seven main functions. Each one exists because the previous step alone cannot give a complete answer.

1. **Active probing.** The client sends numbered UDP probes and records the time each reply arrives. This gives round-trip time, packet loss, and jitter. Without this step there is no evidence that the user traffic is actually affected.
2. **Path tracing.** The trace command reports the IP hops that can be seen. In an SDN, the switches usually do not decrement the IP TTL, so a normal traceroute often shows only one hop. The tool therefore explains this limitation and relies on the flow table for the real path.
3. **Topology discovery.** The topology command asks the controller for switches, links, and hosts. It converts the reply into one graph that every other module can use.

4. **Flow-table inspection.** The flow inspector reads the installed rules and follows the output ports from the source switch to the destination host. This is the authoritative path, not a guess based on the shortest route.
5. **Correlation and localization.** The correlation engine compares probe results with port counters, link state, flow entries, and kernel qdisc counters. It then produces a health verdict, a list of suspected links, a confidence value, and the evidence behind each conclusion.
6. **Backup path selection and rerouting.** When the primary path has failed, the tool ranks the remaining loop-free paths and can install higher-priority OpenFlow rules on the backup route. A dry-run mode shows the rules without changing any switch.
7. **Reporting.** Every run produces JSON files for machines and Markdown, HTML, and PNG files for people. The reports repeat the measurements, the suspected link, the evidence, and the recommended backup path.

2.3 User Classes and Characteristics

- **Network operators and instructors** understand IP addressing, routing, and the basic idea of OpenFlow. They use the CLI to check a path, read the report, and decide whether a reroute is safe.
- **Students and developers** run the tool to learn how the data plane and the control plane relate. They use `make demo`, the scenario scripts, and the pytest suite.
- **Testers and CI runners** do not need to understand the internals. They run the automated commands and compare the output with the expected verdicts in the test plan.

2.4 Operating Environment

SDNScope runs on Linux. The project has been exercised on Ubuntu 22.04, 24.04, and 26.04, including a virtual machine. The supported Python range is 3.10 to 3.14, and the CI workflow tests every version in that range. The latest live run used Python 3.14.4.

Live network emulation needs the following tools:

- Mininet 2.3.0, which creates the hosts, switches, and links.
- Open vSwitch 3.7.1, which provides the virtual switches.
- Ryu 4.34, started with `ryu.app.ofctl_rest`, `ryu.app.rest_topology`, and `ryu.app.simple_switch_13`.
- `iproute2`, for the `tc qdisc netem` delay and loss injection.
- Root or passwordless sudo, because Mininet and `tc` need elevated privileges.

The localhost demo and the unit tests do not need Mininet, Ryu, or root. They run on any normal POSIX system with Python 3 and the packages listed in `requirements.txt`.

2.5 Design and Implementation Constraints

- Mininet and `tc` require root. The tool must therefore separate the privileged scenario runner from the normal diagnostic commands.
- All modules exchange JSON. The field names and types are fixed by `docs/schemas.md`, so a captured fixture and a live response can be used in the same test.
- Target addresses must be valid IPv4 addresses. The client must reject anything else before it creates a socket.
- The current release supports IPv4 only.
- The controller must be started with link discovery enabled. Without it, the topology REST endpoint returns an empty or incomplete link list.

2.6 User Documentation

The repository includes the following documents:

- `README.md`: overview, quick start, requirement mapping, CLI reference, and the latest verified results.
- `SETUP.md`: installation of Python, Mininet, Open vSwitch, and Ryu, including the sudo requirements.
- `docs/DESIGN.md`: internal design, protocol formats, and the formulas used for statistics and confidence.
- `docs/schemas.md`: JSON schemas for topology, probe results, correlation reports, and configuration.
- `docs/EVIDENCE.md`: captured transcripts, comparison tables, and the evidence behind the reported numbers.
- `docs/DEMO.md`: a short demonstration script for a presentation.
- `TEST_CASES.md` and `TESTER_REPORT.md`: the test cases and the tester's written report.
- `SUBMISSION_CHECKLIST.md`: the acceptance checks and the official evidence list.

2.7 Assumptions and Dependencies

The tool assumes that the Ryu controller is reachable over HTTP and that link discovery is enabled. It also assumes that the user has a hosts file or a running discovery mechanism, because a fresh controller may not yet know the hosts. The main software dependencies are `requests` for HTTP, `matplotlib` for the RTT chart, and `pytest` for the tests.

Two limitations are worth stating up front. First, one-way delay figures depend on clock synchronisation between the two hosts; round-trip time does not, because both timestamps are taken on the client. Second, a very short outage can begin and end between two controller polls, so the tool may only see the probe loss, not the link event. In that case the correlation engine falls back to the probe evidence and says so.

3. External Interface Requirements

3.1 User Interfaces

The main interface is the command line. `sdnscope.py` groups the functions into ten subcommands:

Command	What it does
`probe`	Sends UDP probes to a target and writes RTT, loss, and jitter statistics.
`serve`	Starts the UDP echo server, optionally with an artificial delay or drop rate.
`trace`	Runs an IP-layer trace and reports the hops it can actually observe.
`topo`	Asks the controller for switches, links, and hosts and writes a topology file.
`paths`	Builds the graph and lists the loop-free paths between two hosts, shortest first.

Command	What it does
`correlate`	Combines probe results with topology and counter data and writes a diagnosis.
`watch`	Polls the controller and prints link-up and link-down events.
`localize`	Runs the multi-source localization step on its own.
`reroute`	Shows or installs backup flow rules for a failed path.
`report`	Produces the Markdown, HTML, and PNG outputs from a correlation report.

Invalid input is reported as a short error message on standard error, followed by a non-zero exit code. For example, an invalid address is rejected with `[ERROR] Invalid IPv4 address`. The tool does not print a Python traceback for a user input mistake.

3.2 Hardware Interfaces

The software does not drive physical network cards. It works with the virtual Ethernet pairs, network namespaces, and Open vSwitch datapaths that Mininet creates. Link bandwidth, delay, and loss are applied to the virtual switch ports with Linux traffic control (`tc qdisc netem`). On a real testbed the same commands would apply to real interfaces, but that deployment has not been tested.

3.3 Software Interfaces

- **Ryu REST API.** The tool reads `/v1.0/topology/switches`, `/v1.0/topology/links`, `/v1.0/topology/hosts`, `/stats/switches`, `/stats/flow/<dpid>`, and `/stats/port/<dpid>`, and it posts rules to `/stats/flowentry/add`. The default address is `http://127.0.0.1:8080`.
- **Mock controller.** `mock_controller.py` serves the same JSON shapes on a local port. It is used by the demo and by tests that must run without root.
- **JSON artifacts.** The modules read and write `config.json`, `topology.json`, `paths.json`, `diagnostic_result.json`, `trace.json`, and `correlation_report.json`. The exact schema is in `docs/schemas.md`.
- **Build and test interface.** The `Makefile` provides `make install`, `make test`, `make lint`, `make demo`, `make real-check`, and `make real-scenarios`.

3.4 Communications Interfaces

The probe client and server use UDP over IPv4. The default port is 9999. Each probe carries a sequence number and a client timestamp, and the reply repeats them so that the client can calculate the round-trip time.

Controller communication uses HTTP over TCP port 8080 with JSON bodies. OpenFlow 1.3 messages between Open vSwitch and Ryu use TCP port 6653 by default; port 6633 is accepted as the older convention. The probe server binds to localhost or to the Mininet host address, not to a public interface.

4. System Features

The features are grouped in the order in which a diagnostic run uses them: measure, discover, correlate, reroute, and report. Each feature has a short description, a stimulus/response sequence,

and numbered requirements.

4.1 Active Data-Plane Probing and Telemetry

4.1.1 Description and Priority

This feature measures the quality of the path between two hosts. The probe server echoes each request, and the client records the time until the reply arrives. Priority is **high**, because every later conclusion depends on these measurements.

4.1.2 Stimulus and Response

A user or a scenario script starts the server, then runs `sdnscope.py probe` with a target address, a probe count, and a timeout. The client validates the address, sends the probes, waits for replies, and writes `diagnostic_result.json`. If the user also asks for a trace, it writes `trace.json`.

4.1.3 Functional Requirements

- **REQ-1:** The client shall validate the target as an IPv4 address before opening a socket. An invalid value shall produce a clear error and exit code 1.
- **REQ-2:** The server shall echo every probe with its sequence number and timestamp so that the client can compute an accurate round-trip time.
- **REQ-3:** The client shall report the number of probes sent, received, lost, duplicated, and out of order. It shall also report minimum, mean, median, maximum, standard deviation, the 95th percentile, and the RFC 3550 inter-arrival jitter.
- **REQ-4:** The client shall classify the result as healthy, degraded, or failed using the configured thresholds and shall write the complete result as JSON.

4.2 Topology, Path, and Flow Discovery

4.2.1 Description and Priority

This feature answers the question “where did the traffic actually go?”. It reads the controller’s view of the network, builds a graph, lists possible paths, and then checks the flow tables to see which path is really installed. Priority is **high**.

4.2.2 Stimulus and Response

`sdnscope.py topo` reads the controller endpoints and writes `topology.json`. `sdnscope.py paths` reads that file and writes the ranked paths. The flow inspector is called during correlation and reads `/stats/flow/<dpid>` for each switch.

4.2.3 Functional Requirements

- **REQ-5:** The topology module shall read switches, links, hosts, and port state from the controller, or from an explicit hosts file when the controller does not know the hosts yet.
- **REQ-6:** The path module shall build a graph and return all simple loop-free paths between the source and destination, ordered by hop count.
- **REQ-7:** The flow inspector shall reconstruct the active path by following the installed rules and output ports. If the flow table is incomplete, it shall fall back to the graph path and mark the path source as `assumed_shortest` instead of `flow_table`.

4.3 Correlation, Health Classification, and Localization

4.3.1 Description and Priority

This is the part that turns measurements into a diagnosis. It compares the probe result with the control-plane state and with the kernel qdisc counters, then ranks the links that could explain the problem. Priority is **high**.

4.3.2 Stimulus and Response

`sdnscope.py correlate` takes the diagnostic result, the topology, and optional before/after port statistics and qdisc captures. It writes `correlation_report.json`. The report contains the health verdict, the metrics, the suspected links with confidence values, the evidence for each suspicion, and the backup paths.

4.3.3 Functional Requirements

- **REQ-8:** The engine shall classify the path as `FAILED_PATH` when packet loss is at or above the failure threshold (50% by default) or when a link on the active path is missing. It shall classify the path as `DEGRADED_PATH` when there is some packet loss or when the mean RTT is above the degraded limit (50 ms by default). Otherwise the path is `HEALTHY_PATH`.
- **REQ-9:** The engine shall use several independent sources of evidence: OpenFlow port drop counters, topology liveness (a link that disappeared), and Linux kernel qdisc drop counters. A conclusion shall state which sources were used.
- **REQ-10:** The engine shall report a confidence value. A direct kernel qdisc measurement shall be reported separately from the weaker path-membership heuristic, so that a 95% direct measurement is never confused with a 60% guess.

4.4 Backup Path Selection and Rerouting

4.4.1 Description and Priority

When the primary path has failed, this feature selects a loop-free backup and can install the matching OpenFlow rules. Priority is **high** for the selection step; installation is deliberately conservative and is only performed when the user asks for it.

4.4.2 Stimulus and Response

`sdnscope.py reroute` reads the correlation report and the topology, removes the failed links from the graph, ranks the remaining paths, and prints the proposed rules. With `--apply`, it sends the rules to the controller. Without it, the command is a dry run.

4.4.3 Functional Requirements

- **REQ-11:** The reroute helper shall never propose a path that contains a failed or suspect link.
- **REQ-12:** The helper shall support a dry-run mode that changes nothing, and an apply mode that installs higher-priority backup rules with the correct destination address, not the loopback address.

4.5 Reporting

4.5.1 Description and Priority

This feature turns the JSON results into something a person can read and an evaluator can check. Priority is **medium**; the run is still valid without a report, but the report is required for submission.

4.5.2 Stimulus and Response

`sdnscope.py report` reads the correlation report and the diagnostic result and writes `report.md`, `report.html`, and `rtt_timeseries.png` into the run directory.

4.5.3 Functional Requirements

- **REQ-13:** The report shall include the probe statistics, the path, the health verdict, the suspected link with confidence, the evidence, the recommended backup, and the environment in which the run was taken.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

A probe timeout must be respected. The default timeout is 1.0 second per packet, and a run must finish even when every probe is lost; it must not hang.

On the live testbed, link-loss detection and correlation completed within the polling interval, and rerouting restored 0% packet loss on the next probe run. The latest measured runs are:

Scenario	Probes	Loss	Mean RTT	Verdict
Healthy	50 / 50	0.00%	9.68 ms	`HEALTHY_PATH`
Degraded	100 / 92	8.00%	66.33 ms	`DEGRADED_PATH`
Failed	20 / 0	100.00%	–	`FAILED_PATH`
Recovery (after reroute)	50 / 50	0.00%	14.29 ms	`HEALTHY_PATH`

The full pytest suite currently runs in about six seconds on the development machine. That is fast enough to be used as a pre-flight gate before a live run.

5.2 Safety Requirements

The live scenarios change real kernel and OpenFlow state, so they must clean up after themselves. The scenario runner removes the `tc` rules and stops the network at the end of each scenario, even when a step fails. The reroute helper defaults to a dry run, and rules are only installed when `--apply` is present.

The tool is meant for a lab network. It should not be pointed at a production controller without a maintenance window, because it can add or replace flow entries.

5.3 Security Requirements

The probe server and the mock controller bind to localhost or to an isolated Mininet address by default. They are not intended to be exposed to the internet.

User input is validated before use. Target addresses must be valid IPv4 addresses, ports must be in range, and file paths are checked before they are read or written. No password or token is stored in the repository: the default password field in `config.json` is empty, and the controller password can be supplied through the `CONTROLLER_PASSWORD` environment variable.

5.4 Software Quality Attributes

- **Testability.** Every module has unit or integration tests. The suite contains 82 tests and uses both synthetic fixtures and fixtures captured from the live diamond. The measured total coverage is 68%.
- **Reliability.** If the controller is down or returns incomplete data, the tool prints a clear error and exits instead of crashing with an unhandled exception. A missing flow table does not stop the run; the path source is marked as an assumption.
- **Maintainability.** Each stage (probe, discovery, inspection, correlation, reroute, report) is a separate module with a documented JSON interface. A change in one stage does not require rewriting the others.
- **Usability.** The CLI has short help text for every command, and every run leaves a report that explains the verdict in ordinary language.

5.5 Business Rules

1. A path is **healthy** when it has no packet loss and its mean RTT is within the configured limit.
2. A path is **degraded** when it has some packet loss or a mean RTT above the limit, but the links are still present.
3. A path is **failed** when packet loss reaches the failure threshold or a link on the path is missing.
4. Rerouting is only proposed when the primary path has failed and a loop-free alternate path exists.
5. A conclusion must be backed by saved evidence. A run without its JSON files is not accepted as proof.

6. Other Requirements

Every live or demo run is stored in a timestamped directory under `results/`. The directory contains the environment details, the probe result, the topology, the path list, the correlation report, the reports, and, where applicable, the before/after port and flow snapshots. The `results/latest` link points to the most recent live four-scenario run.

Test evidence that is quoted in the documentation is also copied to `TEST_EVIDENCE/`. The evidence files are kept small and text-based where possible so that they can be reviewed without special tools.

Appendix A: Glossary

- **DPID.** Datapath identifier. The unique number that identifies an OpenFlow switch, for example 1, 2, or 3 in the diamond.
- **Mininet.** A tool that builds a virtual network of hosts, switches, and links on one Linux machine.
- **Netem.** The Linux `tc qdisc` module used to add delay or drop packets on a link.
- **OpenFlow.** The protocol that lets a controller read and change switch flow tables.

- **Qdisc.** A kernel queue attached to a network interface. Its drop counter shows packets that the kernel discarded before they left the interface.
- **RTT.** Round-trip time. The time from sending a probe to receiving its reply, measured on the client.
- **Jitter.** The variation in delay between packets. SDNScope reports both a standard deviation and the RFC 3550 inter-arrival estimate.

Appendix B: Analysis Models

The network model is the diamond shown in Section 2.1. Host `h1` connects to `s1`, and host `h2` connects to `s2`. The direct link `s1--s2` is the primary route and gives a three-hop path. When that link is down, traffic can go `h1 -> s1 -> s3 -> s2 -> h2`, which is one hop longer.

The data flow through the tool is:

```
probe client/server -> diagnostic_result.json
controller REST     -> topology.json -> paths.json
flow tables         -> active path
port and qdisc stats -> before/after counters
            |
            v
      correlate.py -> correlation_report.json
            |
            +--> reroute_helper.py -> OpenFlow rules
            |
            +--> report_generator.py -> report.md / .html / .png
```

The confidence model combines the evidence sources. A confirmed link disappearance or a matching direct qdisc drop is strong evidence and is reported with a high confidence. If only the path membership is known (the loss is on the path, but no counter points to one link), the result is reported as a heuristic with a lower confidence, currently 60%.

Appendix C: To Be Determined List

Version 1.1 implements and verifies all core requirements. The following items are intentionally left for later work:

- **TBD-1:** IPv6 probe targets and IPv6 flow match fields.
- **TBD-2:** A long-running daemon mode that polls and reroutes without a manual command.
- **TBD-3:** Larger topologies and multi-controller environments.
- **TBD-4:** Per-segment probe responders on switch attachment points, so that a failure can be narrowed to a single segment without relying on controller polling.

Appendix D: Requirement Traceability

The seven expectations of the course problem statement map to the implementation as follows.

#	Expectation	Main modules
1	UDP diagnostic probes	`probe_server.py`, `diagnostic_client.py`
2	Measure round-trip time	`diagnostic_client.py`
3	Detect packet loss	`diagnostic_client.py`, `probe_server.py`
4	Path analysis	`path_analysis.py`, `flow_inspector.py`
5	SDN topology discovery	`sdn_topology.py`, `topology_graph.py`
6	Correlate measurements with paths	`flow_inspector.py`, `correlate.py`
7	Identify failed paths and reroute	`correlate.py`, `reroute_helper.py`